Identificação de Espécies de Aves pelo Canto com Aprendizagem Profunda: CNNs, Redes Recorrentes e Modelos de Áudio Pré-Treinados

Proposta de Projeto — CIN0040 Introdução à Aprendizagem Profunda

Kaique Souza Bonfim
Centro de Informática (CIn)
Universidade Federal de Pernambuco
Recife, Brasil
ksb3@cin.ufpe.br

Giovanny Lira de Araujo Cunha
Centro de Informática (CIn)
Universidade Federal de Pernambuco
Recife, Brasil
glac2@cin.ufpe.br

Caio Henrique Araujo Braga
Centro de Informática (CIn)
Universidade Federal de Pernambuco
Recife, Brasil
chab@cin.ufpe.br

***Resumo*—Esta proposta descreve um projeto de classificação de espécies de aves a partir de gravações de áudio. Os sinais são convertidos em espectrogramas log-Mel e classificados por quatro famílias de modelos: uma rede convolucional treinada do zero, uma rede convolucional-recorrente (CRNN) e dois modelos pré-treinados em larga escala no AudioSet (PANNs CNN14 e, se o orçamento computacional permitir, o Audio Spectrogram Transformer). Os dados vêm de um subconjunto de 15 a 20 espécies do desafio BirdCLEF. Além de comparar arquiteturas sob validação cruzada agrupada por gravação, o projeto investiga o efeito de técnicas de aumento de dados específicas para áudio e a robustez dos modelos a ruído ambiental, situação típica do monitoramento acústico em campo.**

***Palavras-chave*—bioacústica, classificação de áudio, espectrograma Mel, redes neurais convolucionais, CRNN, transferência de aprendizado, aumento de dados.**

I. Objetivos

A. Objetivo geral

Desenvolver e avaliar comparativamente modelos de aprendizagem profunda para identificar a espécie de uma ave a partir de trechos curtos de sua vocalização, medindo o ganho trazido pela transferência de aprendizado e pelo aumento de dados em condições de ruído.

B. Objetivos específicos

1) Construir um *pipeline* reprodutível no Google Colab que transforme gravações brutas em espectrogramas log-Mel segmentados.
2) Implementar e treinar uma CNN do zero e uma CRNN como modelos de referência.
3) Ajustar (*fine-tuning*) modelos de áudio pré-treinados no AudioSet (PANNs CNN14 e, opcionalmente, AST).
4) Medir o efeito de SpecAugment, *mixup* e mistura de ruído de fundo no desempenho.
5) Avaliar a robustez dos modelos sob diferentes razões sinal-ruído (SNR).

C. Hipóteses

- **H1:** modelos pré-treinados no AudioSet superam os modelos treinados do zero em F1-macro, especialmente nas espécies com poucas gravações.
- **H2:** a modelagem temporal explícita da CRNN supera a CNN simples de mesmo porte.
- **H3:** o aumento de dados específico para áudio reduz a queda de desempenho sob ruído (SNR $\leq$ 10 dB).

II. Justificativa

O Brasil abriga quase duas mil espécies de aves, e o monitoramento acústico passivo, com gravadores deixados em campo, tornou-se uma ferramenta central para acompanhar a biodiversidade [1]. Esses gravadores produzem milhares de horas de áudio, inviáveis de anotar manualmente; sistemas como o BirdNET [2] mostram que redes profundas podem automatizar essa tarefa em escala.

Do ponto de vista da disciplina, o problema reúne em um só projeto conceitos centrais de aprendizagem profunda: redes convolucionais aplicadas a representações tempo-frequência, redes recorrentes para dependências temporais e transferência de aprendizado. Ele traz ainda desafios realistas, como rótulos fracos, desbalanceamento de classes e ruído, que tornam a análise experimental mais rica do que em bases de imagens tradicionais.

III. Metodologia

A Fig. 1 resume o fluxo proposto, detalhado nas subseções a seguir.

A. Base de dados

Será usado o conjunto de treino de uma edição recente do desafio BirdCLEF [3], disponibilizado no Kaggle. Ele é composto por gravações focais de aves, em sua maioria provenientes do repositório colaborativo xeno-canto [4], com rótulo de espécie e metadados como localização e avaliação de qualidade.

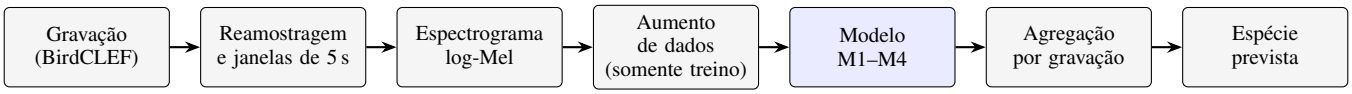


Figura 1. Diagrama de blocos do *pipeline* proposto. M1: CNN do zero; M2: CRNN; M3: PANNs CNN14; M4: AST (opcional).

Para caber nos limites do Colab, será selecionado um subconjunto de 15 a 20 espécies com maior número de gravações, limitado a cerca de 150 gravações por espécie e priorizando as de melhor qualidade. A análise exploratória vai descrever a distribuição de gravações e de duração por espécie, a faixa de frequência típica de cada canto e a proporção de trechos sem vocalização. Como alternativa, caso haja restrição de acesso, um subconjunto equivalente de espécies brasileiras pode ser obtido diretamente pela API pública do xeno-canto.

B. Pré-processamento

As gravações serão convertidas para mono e reamostradas a 32 kHz. Cada gravação é dividida em janelas de 5 s (com sobreposição de 50% no treino), e janelas com energia muito baixa são descartadas para mitigar o problema de rótulos fracos, já que o rótulo vale para a gravação inteira e não para cada trecho.

Para os modelos treinados do zero, cada janela vira um espectrograma log-Mel com 128 bandas entre 50 Hz e 14 kHz. Para os modelos pré-treinados, a extração segue a configuração original de cada um (PANNs: 32 kHz e 64 bandas [5]; AST: 16 kHz e 128 bandas [6]). Os espectrogramas serão pré-computados e salvos no Google Drive para acelerar o treino.

C. Modelos

A Tabela I resume as arquiteturas comparadas.

Tabela I
MODELOS COMPARADOS

ID	Arquitetura	Inicialização
M1	CNN com 4 blocos Conv-BN-ReLU-Pool + *pooling* global [7]	Aleatória
M2	CRNN: blocos convolucionais + GRU bidirecional no eixo do tempo [8]	Aleatória
M3	PANNs CNN14 [5]	AudioSet
M4	Audio Spectrogram Transformer [6] (opcional)	AudioSet

Todos os modelos terminam em uma camada *softmax* sobre as espécies selecionadas e são treinados com entropia cruzada, otimizador AdamW e parada antecipada pelo F1-macro de validação. Na inferência, a predição de uma gravação é a média das probabilidades de suas janelas.

D. Aumento de dados

Serão avaliadas três técnicas, isoladas e combinadas: SpecAugment [9], que mascara faixas de tempo e frequência; *mixup* [10], que combina pares de exemplos e rótulos; e mistura de ruído de fundo com sons ambientais da base ESC-50 [11] (chuva, vento, insetos).

E. Protocolo experimental

Para evitar vazamento de informação, todas as divisões são **agrupadas por gravação**: janelas de uma mesma gravação nunca aparecem em partições diferentes. Um conjunto de teste fixo (20% das gravações, estratificado por espécie) é separado no início e usado apenas na avaliação final. Nos 80% restantes aplica-se validação cruzada estratificada e agrupada com $k = 5$.

- **E1 – Arquiteturas:** M1 a M4 sob a mesma configuração de aumento de dados, em validação cruzada de 5 *folds* (testa H1 e H2).
- **E2 – Ablação de aumento de dados:** sem aumento, cada técnica isolada e todas combinadas, aplicadas ao melhor modelo do zero e ao melhor pré-treinado, em um *fold* fixo com 3 sementes.
- **E3 – Robustez a ruído:** o conjunto de teste é corrompido com ruído da ESC-50 em SNR de 20, 10, 5 e 0 dB, e mede-se a queda de desempenho com e sem aumento de dados (testa H3).
- **E4 – Eficiência de dados:** treino com 25%, 50% e 100% das gravações, comparando modelos do zero e pré-treinados.

F. Métricas e análise

A métrica principal é o **F1-macro**, adequada ao desbalanceamento entre espécies. Também serão reportadas acurácia *top-1* e *top-3*, mAP (padrão em BirdCLEF e AudioSet) e matrizes de confusão. Os resultados são apresentados como média $\pm$ desvio-padrão entre *folds* ou sementes. A análise qualitativa inclui mapas Grad-CAM sobre os espectrogramas, para verificar se o modelo foca no canto ou no ruído, e projeções t-SNE dos *embeddings* para identificar espécies confundidas.

G. Recursos computacionais e bibliotecas

Todo o código será escrito em Python no Google Colab, com GPU NVIDIA T4 (cerca de 15 GB de memória). As principais bibliotecas são PyTorch e torchaudio (modelos e treino), librosa [12] (processamento de áudio), Hugging Face Transformers (AST), o código oficial do PANNs, scikit-learn (divisões e métricas) e matplotlib (visualizações). Para respeitar os limites de sessão do Colab, serão usados espectrogramas pré-computados, precisão mista (*automatic mixed precision*) e *checkpoints* salvos no Drive. O M4 é o modelo mais custoso e será treinado apenas se o orçamento permitir.

IV. RESULTADOS ESPERADOS

Espera-se obter: (i) um *pipeline* reprodutível de classificação de cantos de aves; (ii) uma comparação quantitativa entre CNN, CRNN e modelos pré-treinados; (iii) evidências sobre

o papel do aumento de dados na robustez a ruído; e (iv) uma discussão sobre limitações, como rótulos fracos e confusão entre espécies de canto semelhante.

V. Cronograma de atividades

O projeto se estende de 14/10 a 25/11/2026, dividido em seis semanas (Tabela II).

Tabela II
Cronograma (S1 = 14–20/10; S2 = 21–27/10; S3 = 28/10–03/11; S4 = 04–10/11; S5 = 11–17/11; S6 = 18–25/11)

Atividade	S1	S2	S3	S4	S5	S6
Proposta e revisão bibliográfica	•					
Coleta dos dados e análise exploratória	•	•				
Pipeline de pré-processamento		•				
M1 e M2 com validação cruzada (E1)		•	•			
Fine-tuning de M3 e M4 (E1)			•	•		
Ablação e robustez a ruído (E2, E3)				•	•	
Eficiência de dados (E4) e análises					•	
Relatório IEEE e slides					•	•
Entrega final (25/11)						•

Divisão de tarefas (proposta): o Autor 1 fica responsável pelos modelos treinados do zero (M1 e M2); o Autor 2, pelos modelos pré-treinados (M3 e M4); e o Autor 3, pelos dados, pelo aumento de dados e pelos experimentos de robustez. A análise dos resultados, o relatório e a apresentação são feitos em conjunto.

Referências

[1] D. Stowell, "Computational bioacoustics with deep learning: a review and roadmap," *PeerJ*, vol. 10, e13152, 2022.

[2] S. Kahl, C. M. Wood, M. Eibl, and H. Klinck, "BirdNET: A deep learning solution for avian diversity monitoring," *Ecological Informatics*, vol. 61, 101236, 2021.

[3] S. Kahl *et al.*, "Overview of BirdCLEF 2021: Bird call identification in soundscape recordings," in *Working Notes of CLEF 2021 – Conference and Labs of the Evaluation Forum*, CEUR Workshop Proceedings, 2021.

[4] Xeno-canto Foundation, "xeno-canto: Sharing wildlife sounds from around the world." [Online]. Disponível em: https://xeno-canto.org

[5] Q. Kong, Y. Cao, T. Iqbal, Y. Wang, W. Wang, and M. D. Plumbley, "PANNs: Large-scale pretrained audio neural networks for audio pattern recognition," *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 28, pp. 2880–2894, 2020.

[6] Y. Gong, Y.-A. Chung, and J. Glass, "AST: Audio Spectrogram Transformer," in *Proc. Interspeech*, 2021, pp. 571–575.

[7] S. Hershey *et al.*, "CNN architectures for large-scale audio classification," in *Proc. IEEE ICASSP*, 2017, pp. 131–135.

[8] K. Choi, G. Fazekas, M. Sandler, and K. Cho, "Convolutional recurrent neural networks for music classification," in *Proc. IEEE ICASSP*, 2017, pp. 2392–2396.

[9] D. S. Park *et al.*, "SpecAugment: A simple data augmentation method for automatic speech recognition," in *Proc. Interspeech*, 2019, pp. 2613–2617.

[10] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond empirical risk minimization," in *Proc. ICLR*, 2018.

[11] K. J. Piczak, "ESC: Dataset for environmental sound classification," in *Proc. 23rd ACM Int. Conf. Multimedia*, 2015, pp. 1015–1018.

[12] B. McFee *et al.*, "librosa: Audio and music signal analysis in Python," in *Proc. 14th Python in Science Conf.*, 2015, pp. 18–25.